PRODUCT ARCHITECTURE & TECHNICAL IMPLEMENTATION

# StepUp.

## Technical Architecture

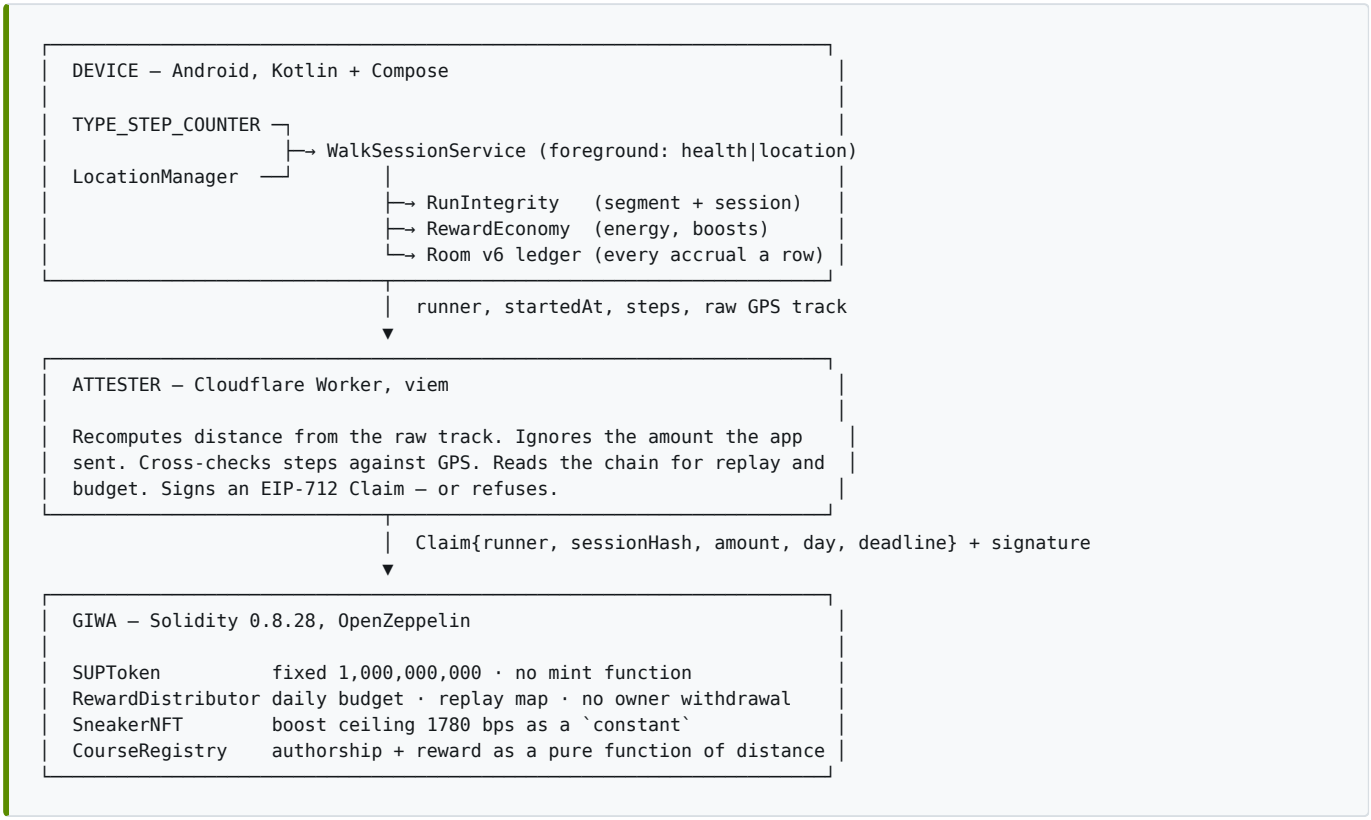
How a run on a phone becomes SUP on GIWA

| | |
|---|---|
| **App** | v1.15.1 · Android · Kotlin + Jetpack Compose |
| **Attester** | Cloudflare Worker · viem · EIP-712 |
| **Chain** | GIWA Sepolia · chain 91342 · 4 contracts live |
| **Repository** | github.com/mycyi1994-hash/GIWASTEPN |

Every number in this document is a constant you can open in the repository.

## 1. System overview

StepUp has three planes, and the split between them is the design.




**The division of labour, stated once:** the attester decides *who gets how much*; the contract decides *how much may exist*. Fraud detection is a moving target and belongs off chain where it can be updated. A supply cap is not, and belongs on chain where it cannot.

---

## 2. Client architecture

| Concern | Choice | Where |
|---|---|---|
| Language / UI | Kotlin 2.0.21, Jetpack Compose + Material 3, Navigation Compose | `app/` |
| SDK | minSdk 26 · targetSdk 35 · compileSdk 35 · JVM target 17 | `app/build.gradle.kts` |
| Pattern | MVVM + Repository, `StateFlow` end to end, no `LiveData` | `ui/screens/*/…ViewModel.kt` |
| DI | Manual — one `object ServiceLocator`, wired in `Application.onCreate` | `core/ServiceLocator.kt` |
| Persistence | Room 2.6.1 via KSP — `strideup.db`, **schema v6, 12 entities, 12 DAOs** | `data/local/` |

| Concern | Choice | Where |
|---|---|---|
| Settings | DataStore ( `UserPrefs` ) — step baselines, energy and streak, locale, daily goal, selected course | `data/prefs/UserPrefs.kt` |
| Domain | Pure Kotlin objects, no Android imports, unit-tested | `domain/` |
| i18n | 665 strings × ko / en / zh / ja + `localeConfig` in-app override | `res/values*/strings.xml` |

**Why manual DI.** One `object` with a dozen `lateinit` fields replaces Hilt's annotation processor, its build-time cost and its lifecycle rules. The graph is one screen long and readable top to bottom; nothing in the app needs scoped injection beyond "one instance per process."

**Room entities (v6):** `DailySteps` · `WalkSession` · `Reward` · `Sneaker` · `Boost` · `ClaimedEvent` · `Crew` · `CrewMembership` · `Post` · `Comment` · `Course` · `Notification`.

The `Reward` table is the ledger. Every accrual and every spend is a row, which is what makes the local balance auditable today and reconcilable against on-chain claims later. `fallbackToDestructiveMigration()` is deliberate for this milestone — while the schema is still moving, rebuilding local demo data beats crashing on a bad migration. It comes out when the schema freezes.

---

## 3. Measurement — from sensor to distance

### 3.1 Steps

`sensor/StepTracker.kt` wraps `TYPE_STEP_COUNTER`, which reports *cumulative steps since boot*. That raw value is useless on its own; three corrections make it a day counter:

- **Day rollover** — at a date change the current cumulative value becomes the new baseline, so today starts at zero without losing the sensor's continuity.
- **Reboot** — a cumulative value below the stored baseline means the counter reset; the baseline drops to zero and today's already-persisted steps are re-applied as an offset instead of being lost.
- **Ordering** — sensor events are funnelled through a `CONFLATED` channel into a single coroutine, so two events can never race on the DataStore baseline.

`hasReading` guards the initial `StateFlow` value of `0`: session accounting only trusts emissions after the first real sensor event, which is what stopped "steps walked before the app was opened" from vanishing (v1.15.1).

### 3.2 GPS and the run session

`service/WalkSessionService.kt` is a foreground service typed `health|location`, so a run survives screen-off and app switching.

| | |
|---|---|
| GPS provider | `requestLocationUpdates(GPS_PROVIDER, 2500 ms, 6 m)` |

| | |
|---|---|
| Fallback | `NETWORK_PROVIDER, 4000 ms, 10 m` when GPS is disabled |
| Control | `ACTION_START` / `PAUSE` / `RESUME` / `STOP` intents |
| UI binding | `WalkSessionService.state: StateFlow<WalkSessionState>` on the companion — screens observe state, never the service |
| Party | Party size arrives as an intent extra; the multiplier is applied at settlement |

### 3.3 Distance and the course map

Distance is **Haversine per segment** (R = 6,371,000 m), summed only over segments that pass the integrity check in § 4. Rendering normalizes the polyline with a `cos(latitude)` correction so a course drawn at Seoul's latitude is not horizontally stretched, simplifies it, and draws it on a `Canvas` over OSM tiles — start dot, finish flag, live runner dot, automatic lap split every kilometre plus a manual lap button.

---

## 4. Run integrity — proving it was a run

`domain/RunIntegrity.kt`. A step sensor cannot tell a runner from a phone rattling in a car, so GPS segment speed is judged alongside it. Two layers plus one independent axis:

| Layer | Rule | Constant |
|---|---|---|
| **Segment** | Above human speed → the segment's distance is discarded entirely and never drawn | `MAX_SPEED_KMH = 25.0` |
| **Segment (noise floor)** | Under 5 m or under 1 s is **not judged** — punishing GPS jitter costs honest runners | `MIN_SEGMENT_METERS = 5.0`, `MIN_SEGMENT_SEC = 1` |
| **Session** | ≥3 flagged segments *and* flagged ratio ≥ 0.5 → `VOID`, no reward | `MIN_FLAGS_FOR_VOID = 3`, `VOID_FLAG_RATIO = 0.5` |
| **Cadence** | Over 240 steps/min after a 60 s grace window → `VOID` regardless of GPS | `MAX_CADENCE_SPM = 240.0`, `CADENCE_GRACE_SEC = 60` |

`verdict()` returns `CLEAN` · `FLAGGED` · `VOID`; only `VOID` blocks accrual. 25 km/h sits above any real runner (marathon WR pace ≈ 20.9 km/h, sampled over 2.5 s windows) and below cycling and driving. Every function here is pure and covered by unit tests that run in CI.

---

## 5. Reward economy

`domain/RewardEconomy.kt` — pure functions, no Android, unit-tested.

```
earnableSteps  = floor(energy × 600 / energyEfficiency)
rewardedSteps  = min(walkedSteps, earnableSteps)
points         = rewardedSteps × 0.01 × sneaker × party × boost
energyUsed     = rewardedSteps × energyEfficiency / 600
```

| Parameter | Value |
|---|---|
| Accrual | **0.01 SUP per step, only during an active run session** |
| Energy | 1 cell = **600 rewardable steps**, 10 cells base, +2 per sneaker level, refilled at midnight. At 0 energy accrual stops entirely |
| Sneaker boost | `rarity + variant × 0.3% + (level − 1) × 0.5%` → **max +17.8%** (Legendary variant 1, Lv.30) |
| Comfort | Up to **15% less energy** per step, clamped to `[0.5, 1.0]` |
| Party run | `1 + 0.10 × min(size − 1, 5)` → **×1.5 max** |
| XP booster | ×2 for 24 h, costs 200 SUP |
| Course completion | `km × 1.0 SUP`, capped at 42, paid at ≥98% coverage |
| Sinks | Mint 500 · upgrade `level × 100 × (1 + rarity × 0.25)` · boosts 50–200 |

**The ceiling is the thesis.** The strongest possible sneaker earns **+17.8%** over a free one — not 3×, not 10×. Capital cannot outrun legs, and that is enforced by a `constant` in three codebases rather than by an operator's promise. Full model and per-persona balance tables: TOKENOMICS.md.

---

## 6. Attester — the off-chain judge

`attester/` — one Cloudflare Worker, `viem`, 7 passing tests, free tier covers 100k requests/day.

**It does not trust the client.** The app is installable, patchable and therefore forgeable; the amount it reports is not read at all. The Worker recomputes everything from the raw GPS polyline.

`POST /claim` runs, in order:

1. **Shape** — valid address; finite timestamps; `elapsed ≥ 60 s`; `0 < steps ≤ 48,000`; track has ≥2 points; `endedAt` not in the future.
2. **Recompute** — `inspectTrack()` walks the polyline with the *same* Haversine and the *same* thresholds as the client, then `verdict()`. `VOID` → HTTP 422, no signature, nothing to submit.
3. **Cross-check** — `steps × 0.762 m` against GPS-measured distance. Outside `[0.5×, 2.0×]` one of the two was fabricated → rejected.
4. **Payout** — computed server-side in integer math at 1e18 scale so no float ever reaches the chain; boost clamped to `MAX_BOOST_BPS = 1780`, party capped.

5. **Session hash** — `keccak256("runner|startedAt|steps|round(distanceM)")`, **rebuilt by the server**, never accepted from the client. This is the key the contract dedupes on.
6. **Chain state** — `sessionClaimed(hash)` → 409 if already paid; `dayRemaining(day)` → 429 if today's budget is spent (so no user ever pays gas for a claim that would revert).
7. **Sign** — EIP-712 `Claim(address runner, bytes32 sessionHash, uint256 amount, uint64 day, uint256 deadline)`, domain `StepUpRewards / 1 / chainId 91342 / verifyingContract`, **deadline now + 10 minutes** so a stolen signature has a short life.

The signing key lives as a Worker Secret in a wallet distinct from the deployer. Its blast radius is bounded by § 8.

---

## 7. Contracts on GIWA

Solidity **0.8.28**, OpenZeppelin, Hardhat, **43 passing tests**. All four deployed to **GIWA Sepolia (chain 91342) on 2026-07-31**, reward pool funded with **50,000,000 SUP** — `contracts/deployments/giwaSepolia.json`.

| Contract | Address | What it guarantees |
|---|---|---|
| **SUPToken** | `0xb052A8f6…A9006c1B` | ERC-20 + Burnable + Permit. `TOTAL_SUPPLY = 1,000,000,000`, minted once in the constructor. **No mint function, no minter role** — supply can only fall, through the burns that mints, upgrades and boosts perform |
| **SneakerNFT** | `0x8174f905…BabEFc960` | ERC-721 + Enumerable + Royalty (500 bps). `boost = rarityBps + variant × 30 + (level − 1) × 50`; the maxed Legendary is **1780 bps, a `constant`**. Mint burns 500 SUP from `msg.sender` and requires an EIP-712 `MintAuth` from `roller` with a per-account nonce; **upgrades need no signature** because cost and effect are deterministic |
| **RewardDistributor** | `0x9f9E87bD…aCFE36E1` | EIP-712 claims against a hard **daily budget**: 250,000 SUP/day, halving every 730 days, the series converging to 365,000,000. 7-day claim window, `sessionClaimed` replay map, `Pausable`. **No sweep, no rescue, no owner withdrawal** — SUP leaves only through `claim`, only to a runner |
| **CourseRegistry** | `0x6c815DF0…C588542` | Authorship of a course, on the record. `rewardFor` is a pure function of distance — `SUP_PER_KM = 1`, `MAX_REWARD = 42`, `MIN_DISTANCE_M = 300`, `COMPLETION_THRESHOLD_BPS = 9800`. The polyline stays off chain; `polylineHash` is stored so the served track can be checked |

**Anyone may submit a claim, and the SUP always goes to `c.runner`.** That is deliberate: a runner who has never touched a chain should not need ETH to receive their first reward, so a relayer can pay the gas without being able to redirect the payout.

---

## 8. Trust model — what breaks if a key leaks

| Decision | Made where | Why there |
|---|---|---|
| Who ran, and how much they earn | Attester (off chain) | Fraud rules must change as abuse changes |
| **How much SUP may exist** | `RewardDistributor` (on chain) | A cap that can be edited is not a cap |
| What a sneaker earns | `SneakerNFT` constants | Earning power must be publicly verifiable, not asserted |
| What a course pays | `CourseRegistry.rewardFor` | A runner can compute their reward before starting |

**If the attester key is stolen outright:** the thief can misdirect *one day's emission budget*. They cannot mint SUP, cannot exceed the daily budget, cannot claim a session twice, and cannot drain the pool — every claim is charged against `dailyBudget(day)` and the pool has no withdrawal path. The owner rotates the key with `setAttester` and can `pause` while doing it. That bounded blast radius is precisely what makes it acceptable to keep the judge off chain.

The privileged surface is the whole list: `setAttester`, `pause` / `unpause`, `setRoller`, `setRecorder`, `setBaseURI`, `setRoyaltyReceiver`. There is no third-party-funds path anywhere in it.

---

## 9. One economy, three codebases, kept in sync by tests

The same constants exist in Kotlin, JavaScript and Solidity. If they ever diverge, the app shows one number and the chain pays another — the kind of bug a user notices first. So each side is pinned by tests:

| Codebase | Holds | Tests |
|---|---|---|
| `app/src/main/java/…/domain/` | `RewardEconomy.kt`, `RunIntegrity.kt` | **23 Android unit tests** — `./gradlew :app:testDebugUnitTest` |
| `attester/src/economy.js` | Mirror of both, explicitly documented as a mirror | **7 tests** — `npm test` |
| `contracts/contracts/*.sol` | `MINT_COST`, `VARIANT_BPS`, `LEVEL_BPS`, `STEPS_PER_ENERGY`, `SUP_PER_KM`, `MAX_REWARD` | **43 tests** — `npx hardhat test`, asserting the on-chain constants equal the client's, that the boost ceiling really is 1780 bps, and that the reward pool has no owner withdrawal path |

---

## 10. End-to-end: a run becomes a claim

```
1  User taps START RUN
2  WalkSessionService goes foreground (health|location); sensors attach
3  Every GPS fix   → Haversine segment → RunIntegrity.isPlausible?
                     ├─ no  → distance discarded, segment flagged, not drawn
                     └─ yes → distance accumulated, track drawn, top speed updated
4  Every step tick → RewardEconomy.sessionReward(steps, energy, sneaker, party, boost)
5  User taps STOP  → RunIntegrity.verdict(valid, flagged, steps, elapsed)
                     ├─ VOID → session saved, reward zero, user told why
                     └─ else → SUP credited to the Room ledger, energy debited
6  [next milestone] POST /claim {runner, startedAt, endedAt, steps, boostBps, partySize, track}
7  Attester recomputes, cross-checks, reads the chain, signs EIP-712 or refuses
8  RewardDistributor.claim(Claim, signature)
      → signer == attester?  sessionHash unused?  within 7 days?
      → amount ≤ dayRemaining(day)?  ≤ poolBalance()?
      → SUP transferred to the runner, session marked claimed, Claimed emitted
```

Steps 1–5 are shipped and running on the APK today. Steps 6–8 are written, tested and deployed as contracts; the wiring inside the app is milestone M3.

---

## 11. Build, release and verification

**Android CI** — `.github/workflows/build-apk.yml`, on every push:

```
:app:testDebugUnitTest  →  :app:assembleDebug  →  signature fingerprint check  →  apk-dist
```

The fingerprint step compares the APK's certificate against the committed debug keystore with `keytool`. A CI runner is a fresh machine, so an auto-generated keystore would sign every build differently and every update would fail on-device with `INSTALL_FAILED_UPDATE_INCOMPATIBLE`. A fixed keystore plus a hard check in CI means yesterday's install can always be updated in place.

**Contracts** — Hardhat; `scripts/deploy.js` writes `deployments/giwaSepolia.json`, and Blockscout verification is scripted three ways (`verify.js`, `standard-json.js`, `split-sources.js`) because explorers differ on how they accept multi-file sources.

**Verify this document yourself:**

| Claim | Command |
|---|---|
| Economy and integrity constants hold | `./gradlew :app:testDebugUnitTest` |
| Contract constants match the client | `cd contracts && npm i && npx hardhat test` |
| Attester mirrors the client | `cd attester && npm i && npm test` |
| The app is real | Install the APK on any Android 8.0+ phone |
| The contracts are real | Open any address in § 7 on `sepolia-explorer.giwa.io` |

**Scale:** 81 Kotlin source files · 23 navigation routes over 26 screen composables · 665 strings × 4 languages · Room schema v6 · 44 sneaker designs · 100 achievements · 60 runner levels · 73 tests across three codebases.

---

## 12. What is not done, stated plainly

- **The app still settles locally.** Rewards are written to the Room ledger. The app does not yet connect a wallet or call `claim` — that is milestone M3.
- **The attester is written and tested but not deployed.** `wrangler.toml` already points at the live `RewardDistributor`; publishing it is a five-minute step that has not been taken.
- **NFT metadata is a placeholder.** `baseURI` is `ipfs://REPLACE_WITH_CID/` until the 44 designs are pinned.
- **Community, ranking and course sharing are local + seeded.** There is no backend yet; that is M4.
- **No external audit.** 43 tests are not an audit, and we do not present them as one.

| Milestone | Scope |
|---|---|
| **M1 — done** | Full client: run tracking, courses, NFTs, community, i18n, CI, installable APK |
| **M2 — done** | Four contracts deployed to GIWA Sepolia, 50M SUP pool funded, 43 tests. Remaining: source verification on the explorer |
| **M3** | Wallet connect + on-chain claim in the app; attester published |
| **M4** | Backend for community, ranking and course sharing |
| **M5** | NFT marketplace (trade / rent), Health Connect, decentralised attestation |

---